MINI-PROJET 8

Comparaison des Réseaux de Neurones vs Méthodes Classiques

Réalisé par :

Bensaadoun Aymane

Genin Amine

Table of Contents

1. Introduction 3

2. Démarche Méthodologique 3

3. Modèles Utilisés et Implémentation 4

4. Résultats et Comparaison des Modèles 8

5. Analyse Graphique des Performances, Complexité et Robustesse 10

5. Discussion critique et conclusions 13

1. Introduction

Le Machine Learning est un domaine de l'intelligence artificielle qui permet aux systèmes informatiques d'apprendre à partir des données afin de réaliser des prédictions ou de prendre des décisions. Parmi les différentes tâches d'apprentissage, la classification supervisée occupe une place importante, notamment pour prédire une variable cible à partir de plusieurs caractéristiques.

Dans ce mini-projet, nous appliquons des méthodes de classification au dataset Titanic, dont l'objectif est de prédire si un passager a survécu ou non au naufrage en fonction de ses caractéristiques :

- **Pclass** (classe du passager)
- **Sex** (sexe)
- **Age** (âge)
- **SibSp** (nombre de frères/époux à bord)
- **Parch** (nombre de parents/enfants à bord)
- **Fare** (prix du billet)
- **Embarked** (port d'embarquement)

1.1 Idée du projet

L'idée principale du projet est d'implémenter plusieurs modèles de classification et d'évaluer leur capacité à prédire la variable « Survived ». Ce travail permet de mettre en pratique les étapes essentielles du Machine Learning, notamment le prétraitement des données, l'entraînement des modèles et l'évaluation des performances.

1.2 Objectif de la comparaison

L'objectif est de comparer les performances des méthodes classiques (KNN, Régression Logistique, SVM) avec un modèle basé sur les Réseaux de Neurones (MLP). Cette comparaison permet d'analyser les différences en termes de précision et de robustesse, afin d'identifier le modèle le plus adapté au dataset étudié.

2. Démarche Méthodologique

Dans ce projet, le dataset Titanic a été prétraité en traitant les valeurs manquantes, en encodant les variables catégorielles et en normalisant les données. La normalisation

permet de mettre les variables sur une même échelle afin d'améliorer les performances des modèles. Les données ont ensuite été divisées en ensembles d'entraînement et de test.

Les modèles ont été évalués à l'aide des métriques suivantes :

- **Accuracy** : proportion totale de prédictions correctes.
- **Precision** : proportion de prédictions positives correctes.
- **Recall** : capacité à détecter les observations réellement positives.
- **F1-score** : compromis entre precision et recall.

3. Modèles Utilisés et Implémentation

Dans ce projet, quatre modèles de classification ont été implémentés afin de comparer les performances des méthodes classiques et des réseaux de neurones. Chaque modèle est intégré dans un `Pipeline` comprenant une étape de normalisation suivie du modèle lui-même. Cette approche garantit un prétraitement cohérent pour tous les algorithmes.

3.1 K-Nearest Neighbors (KNN)

Le modèle KNN est une méthode basée sur la proximité entre les observations. Pour prédire la classe d'un individu, l'algorithme considère les *k* voisins les plus proches et attribue la classe majoritaire.

```python
models = {
    "KNN (k=5)": Pipeline([
        ("scaler", StandardScaler()),
        ("model", KNeighborsClassifier(n_neighbors=5))
    ]),
```

Figure 1 : Extrait du code Python pour l'implémentation du modèle KNN.

Dans cette implémentation :

- `StandardScaler()` est utilisé pour normaliser les variables, car KNN est sensible à l'échelle des données.
- `n_neighbors = 5` définit le nombre de voisins considérés.
- `Pipeline` permet d'enchaîner automatiquement la normalisation et le modèle.

Ce choix assure une meilleure stabilité et une comparaison équitable avec les autres méthodes.

3.2 Support Vector Machine (SVM)

Le SVM est un modèle qui cherche à construire une frontière de décision optimale séparant les classes en maximisant la marge entre elles.

```python
"SVM (RBF)": Pipeline([
        ("scaler", StandardScaler()),
        ("model", SVC())
    ]),
```

Figure 2 : Extrait du code Python pour l'implémentation du modèle SVM.

Dans notre cas :

- `StandardScaler()` est appliqué avant l'entraînement.
- `SVC()` utilise par défaut un noyau RBF (si aucun kernel n'est spécifié).
- Le paramètre `C` contrôle le compromis entre la marge et les erreurs de classification.

Le SVM est généralement performant lorsque les données sont bien séparables dans un espace transformé.

3.3 Régression Logistique

La régression logistique est un modèle probabiliste qui estime la probabilité qu'une observation appartienne à une classe donnée. Elle produit une sortie comprise entre 0 et 1, interprétée comme une probabilité.

```python
"LogReg": Pipeline([
        ("scaler", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000))
    ]),
```

Figure 3 : Extrait du code Python pour l'implémentation de la Régression Logistique.

Dans cette implémentation :

- `max_iter = 1000` permet d'assurer la convergence du modèle.
- La normalisation est appliquée afin d'améliorer la stabilité numérique.

Ce modèle constitue une référence simple et efficace pour les problèmes de classification binaire.

3.4 Réseau de Neurones (MLP)

Le Multi-Layer Perceptron (MLP) est un réseau de neurones artificiels composé de plusieurs couches cachées. Il permet de modéliser des relations non linéaires entre les variables.

```python
"NeuralNet (MLP)": Pipeline([
        ("scaler", StandardScaler()),
        ("model", MLPClassifier(
            hidden_layer_sizes=(64, 32),
            activation="relu",
            max_iter=400,
            random_state=RANDOM_STATE
```

Figure 4 : Extrait du code Python pour l'implémentation du modèle MLP.

Les principaux paramètres utilisés sont :

- `hidden_layer_sizes = (64, 32)` : deux couches cachées de 64 et 32 neurones.
- `activation = 'relu'` : fonction d'activation non linéaire.
- `max_iter = 400` : nombre maximal d'itérations d'apprentissage.
- `random_state` : garantit la reproductibilité des résultats.

La normalisation est essentielle pour assurer une convergence correcte du réseau.

4. Résultats et Comparaison des Modèles

Après l'entraînement des différents modèles, leurs performances ont été évaluées sur l'ensemble de test. Les métriques utilisées sont l'accuracy, la precision, le recall et le F1-score. Les temps d'entraînement et de prédiction ont également été mesurés afin d'analyser la complexité de chaque approche.

4.1 Tableau des resultats

Modèle	Accuracy	Precision	Recall	F1-score	Temps entraînement (s)	Temps prédiction (s)
KNN (k=5)	0.8156	0.8000	0.6957	0.7442	0.0069	0.0051
SVM (RBF)	0.8101	0.8431	0.6232	0.7167	0.0174	0.0079
Régression Logistique	0.8045	0.7931	0.6667	0.7244	0.0082	0.0018
Neural Network	0.7933	0.7759	0.6522	0.7087	1.2299	0.0022

4.2 Analyse Comparative

En termes de performance, le modèle KNN obtient la meilleure accuracy et le meilleur F1-score. Le SVM présente une très bonne précision, mais un recall plus faible. La régression logistique offre des résultats équilibrés et stables. Le MLP obtient des performances légèrement inférieures.

Concernant la complexité, le MLP est le modèle le plus coûteux en temps d'entraînement, tandis que les méthodes classiques sont beaucoup plus rapides.

En résumé, pour le dataset Titanic, les modèles classiques offrent un meilleur compromis entre performance et complexité, tandis que le réseau de neurones n'apporte pas d'amélioration significative dans ce cas.

4.3 Tableau Comparatif Global

Modèle	Performance	Complexité	Robustesse
KNN	Élevée	Faible à moyenne	Bonne
SVM	Élevée	Moyenne	Moyenne
Régression Logistique	Moyenne à élevée	Faible	Bonne
Neural Network	Moyenne	Élevée	Moyenne

5. Analyse Graphique des Performances, Complexité et Robustesse

5.1 Analyse des métriques principales

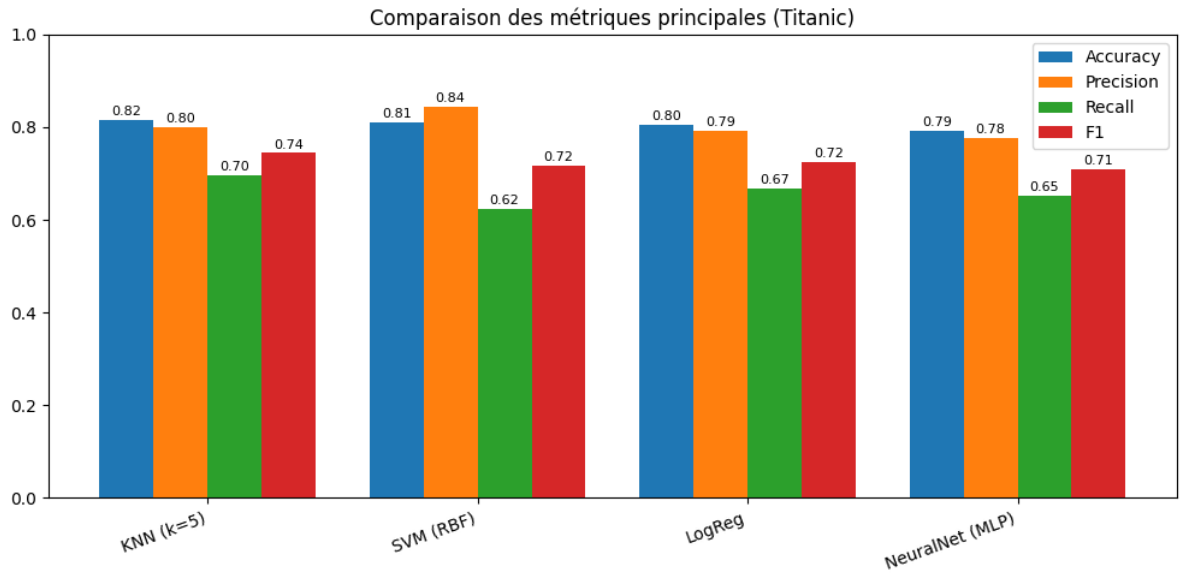

Figure 5 : Comparaison des métriques de performance (Accuracy, Precision, Recall, F1-score)

Ce graphique montre la comparaison des principales métriques pour les quatre modèles. Le KNN obtient la meilleure accuracy et le meilleur F1-score, ce qui indique un bon équilibre dans la détection des passagers survivants et décédés. Le SVM présente

la meilleure precision, ce qui signifie qu'il fait peu de fausses prédictions positives. La régression logistique offre des résultats stables et équilibrés. Le réseau de neurones (MLP) affiche des performances légèrement inférieures aux méthodes classiques.

5.2 Analyse du temps d'entraînement et de prédiction

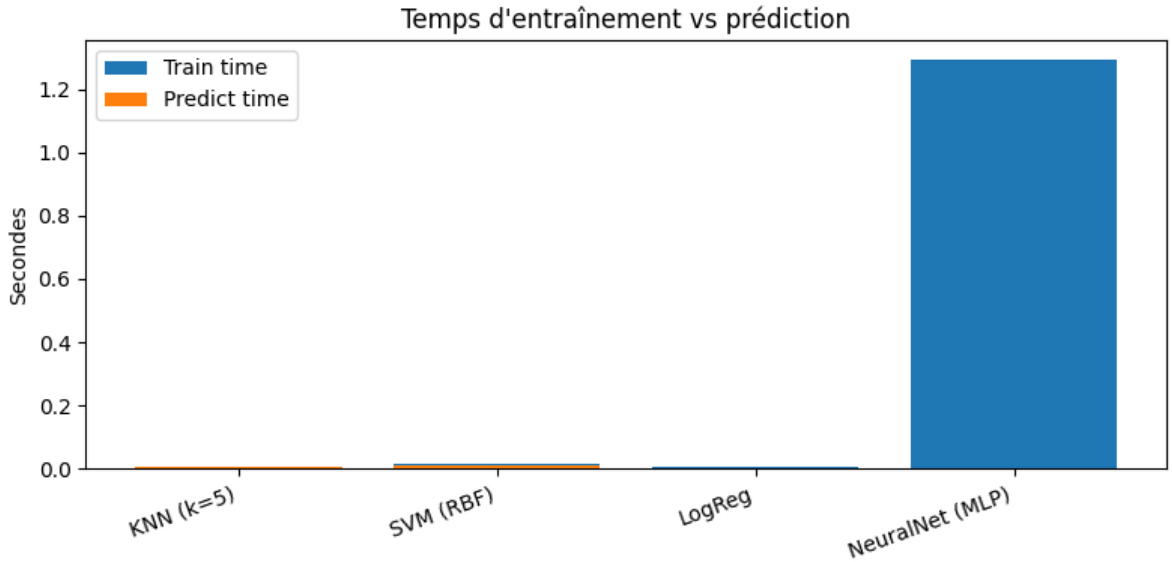

Figure 6 : Comparaison des temps d'entraînement et de prédiction

Ce graphique met en évidence la différence de complexité computationnelle entre les modèles. Le réseau de neurones (MLP) nécessite un temps d'entraînement nettement plus élevé que les autres méthodes. En revanche, les modèles classiques sont rapides, notamment la régression logistique en phase de prédiction. Cela montre que, pour ce dataset, les méthodes classiques sont plus efficaces en termes de temps de calcul.

5.3 Analyse de la sensibilité au bruit

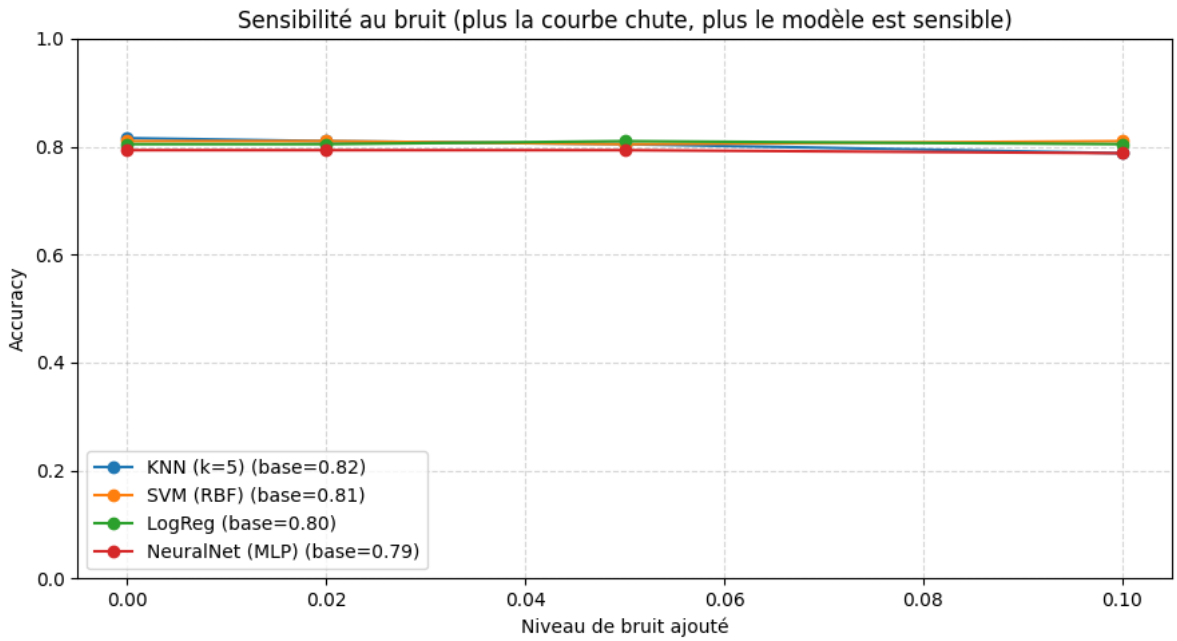

Figure 7 : Sensibilité des modèles au bruit

Ce graphique illustre l'évolution de l'accuracy lorsque du bruit est ajouté aux données. On observe que les performances des modèles diminuent légèrement lorsque le niveau de bruit augmente. Toutefois, la baisse reste faible, ce qui indique une bonne stabilité globale. Le KNN et la régression logistique conservent des performances relativement constantes. Le MLP ne montre pas d'amélioration particulière face au bruit.

Cela suggère que, dans le cas du dataset Titanic, les modèles classiques sont suffisamment robustes et ne sont pas fortement affectés par de petites perturbations des données .

5. Discussion critique et conclusions

Les résultats obtenus montrent que, pour le dataset Titanic, les méthodes classiques de classification offrent de très bonnes performances. Le modèle KNN présente le meilleur équilibre global entre accuracy, F1-score et stabilité. Le SVM se distingue par une excellente precision, tandis que la régression logistique fournit des résultats équilibrés avec une faible complexité.

Le réseau de neurones (MLP), bien que théoriquement plus puissant, n'apporte pas d'amélioration significative dans ce contexte. Son temps d'entraînement est plus élevé sans gain notable en performance. Cela s'explique probablement par la taille modérée du dataset et la nature tabulaire des données, qui ne nécessitent pas un modèle complexe.

En conclusion, ce projet montre que les méthodes classiques restent très efficaces pour des données structurées comme Titanic. Le choix du modèle doit donc dépendre non seulement de la performance, mais aussi de la complexité et du contexte du problème.